



Universidad Autónoma de Baja California

Programación Orientada a Objetos

Unidad 1. Abstracción y encapsulamiento

Abstracción y creación de objetos

Profesor: MC. Itzel Barriba Cázares

Estructura general de una clase

- El contenido de la clase, que son los campos y métodos, se escribirán dentro de llaves. El formato general de una definición es:

```
class NombreClase {  
    tipo_dato instanceVariable1;  
    tipo_dato instanceVariable2;  
    // ...  
    tipo_dato instanceVariable2;  
  
    tipoMetodo nombreMetodo(lista_parametros) {  
        // Cuerpo del metodo  
    }  
    tipoMetodo nombreMetodo2(lista_parametros2) {  
        //Cuerpo del metodo  
  
    }  
    // ...  
    tipoMetodo nombreMetodoN(lista_parametros3) {  
        // Cuerpo del metodo  
    }  
}
```

- En términos generales, los campos y métodos que pertenecen a una clase se denominan miembros de la clase

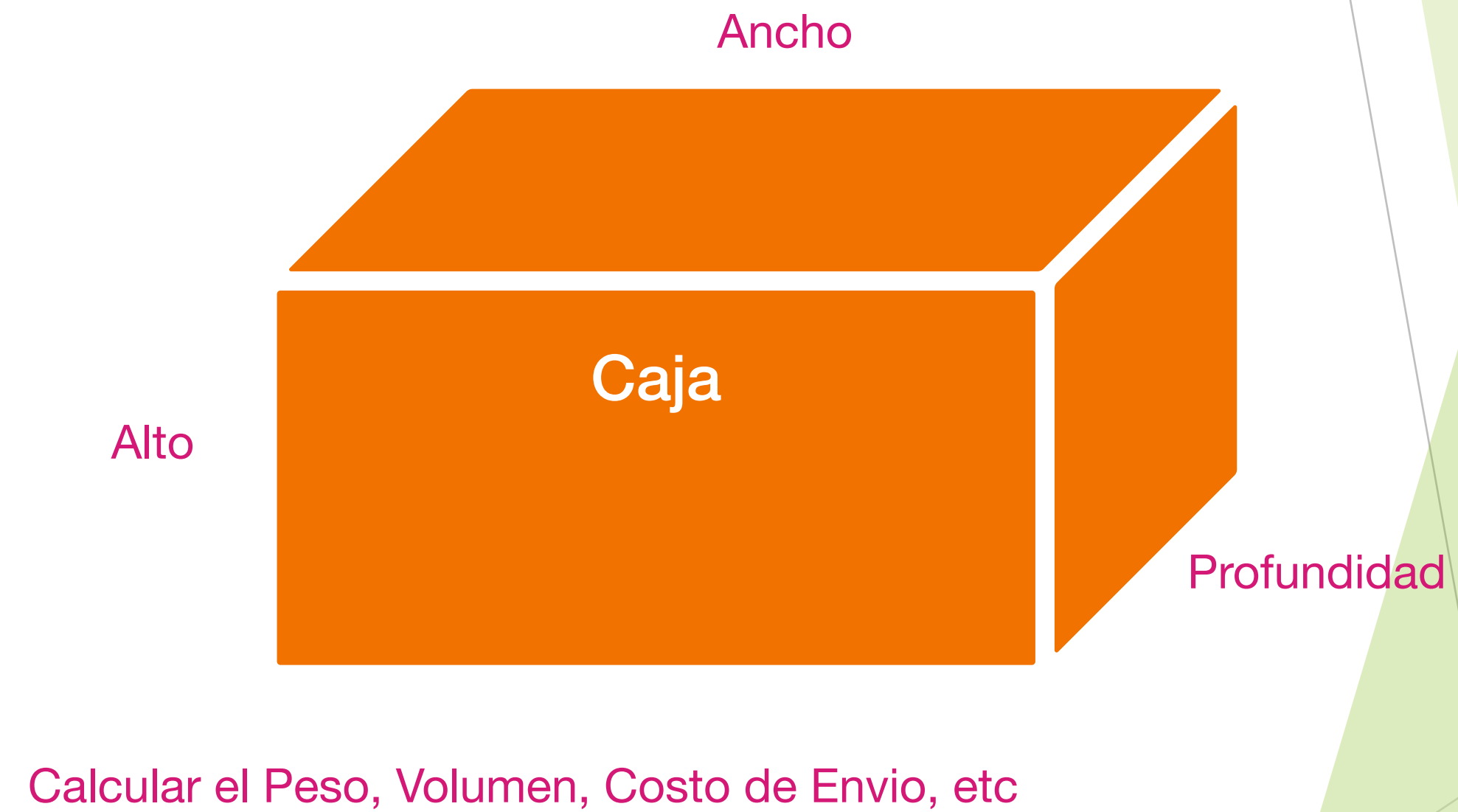
Ejemplo: Definir una clase

- Primero que nada, se tiene que definir los conceptos, los blueprint, plantilla y tipos de datos que se van a usar en la aplicación. son las clases.

```
class NombreClase {  
    // cuerpo de la clase  
  
    //Definición de Atributos  
  
    //Definición de Métodos  
}
```

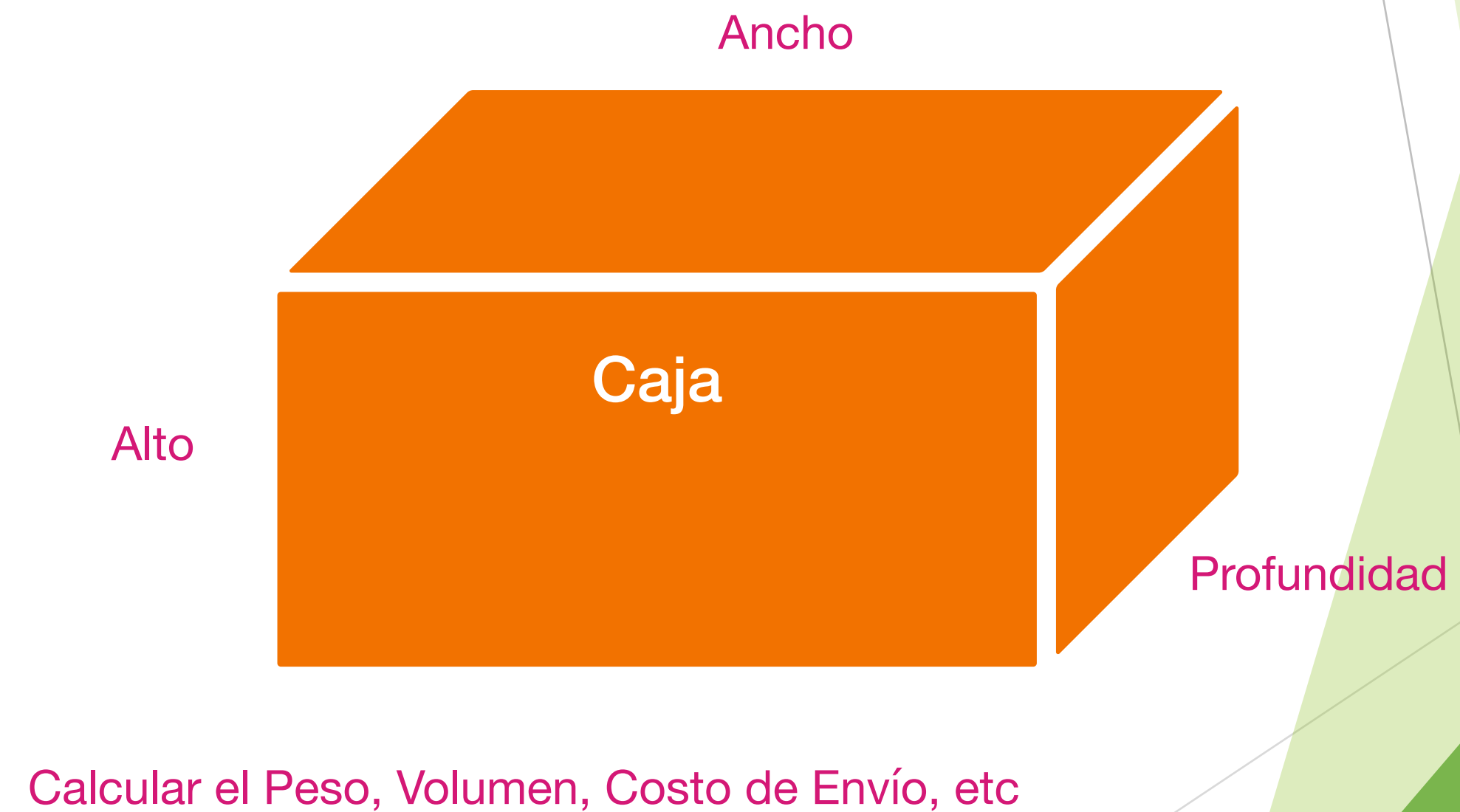

Representación de un objeto Caja

- Una Caja ¿que atributos tiene?



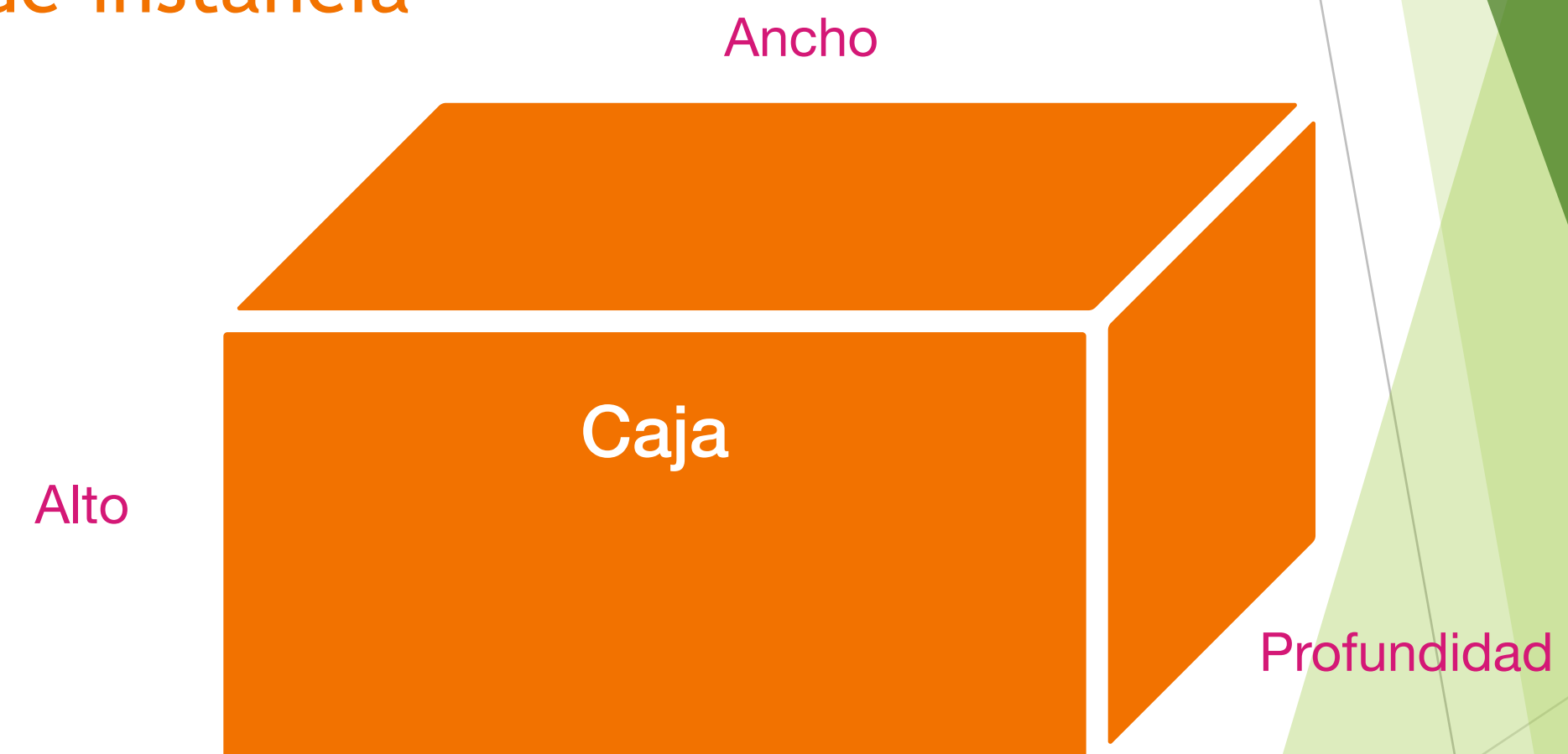
Representación de un objeto Caja

- ▶ Una Caja ¿que atributos tiene?
 - ▶ Los datos, o variables se denominan **variables de instancia**



Representación de un objeto Caja

- ▶ Una Caja ¿que atributos tiene?
 - ▶ Los datos, o variables se denominan **variables de instancia**
- ▶ Una Caja ¿que métodos puede tener?
 - ▶ El código se encuentra dentro de los métodos.



Calcular el Peso, Volumen, Costo de Envío, etc

- ▶ En la mayoría de las clases, los métodos definidos para esa clase actúan sobre las variables de instancia y acceden a ellas.

Variable de instancia

- ▶ Es una variable que está definida en una clase. También se conocen como variables **miembro** o **campos** de una clase.
- ▶ Las variables de instancia pertenecen a objetos, lo que significa que cada objeto de la clase **guarda una copia** separada de esta variable.
- ▶ Cada variable de instancia que defina tiene un valor predeterminado en particular, según el tipo de variable.

Definición de una clase

// Este programa incluye un método dentro de la clase caja

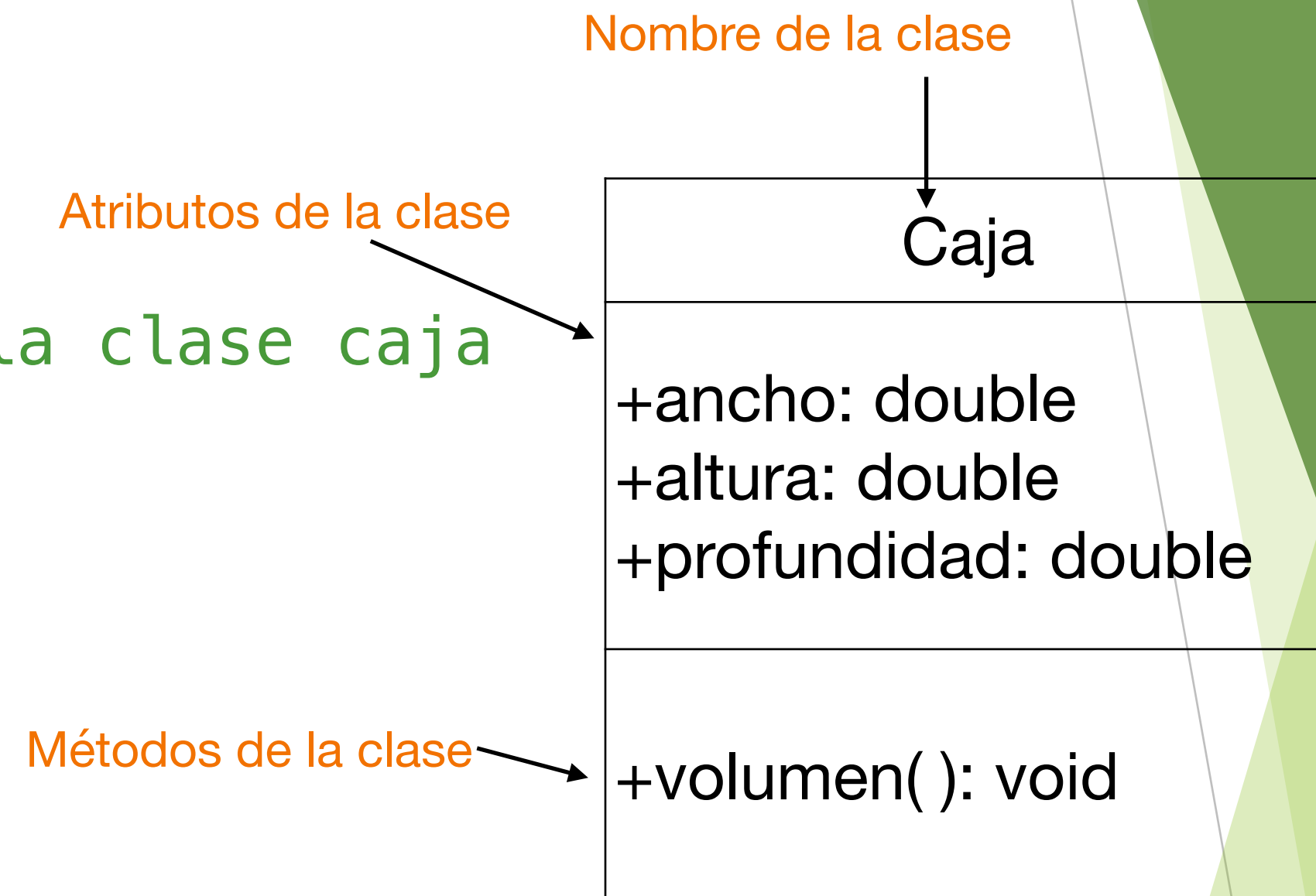
```
public class Caja {  
    double ancho;  
    double alto;  
    double profundidad;
```

// Despliega el volume de una box

```
void volumen() {  
    System.out.print("Volumen es ");  
    System.out.println(ancho * alto * profundidad);  
}
```

```
}
```

- En conjunto, los métodos y las variables definidos dentro de una clase se denominan miembros de la clase.



Crear objetos de la clase caja

- Crear objetos de la clase Caja

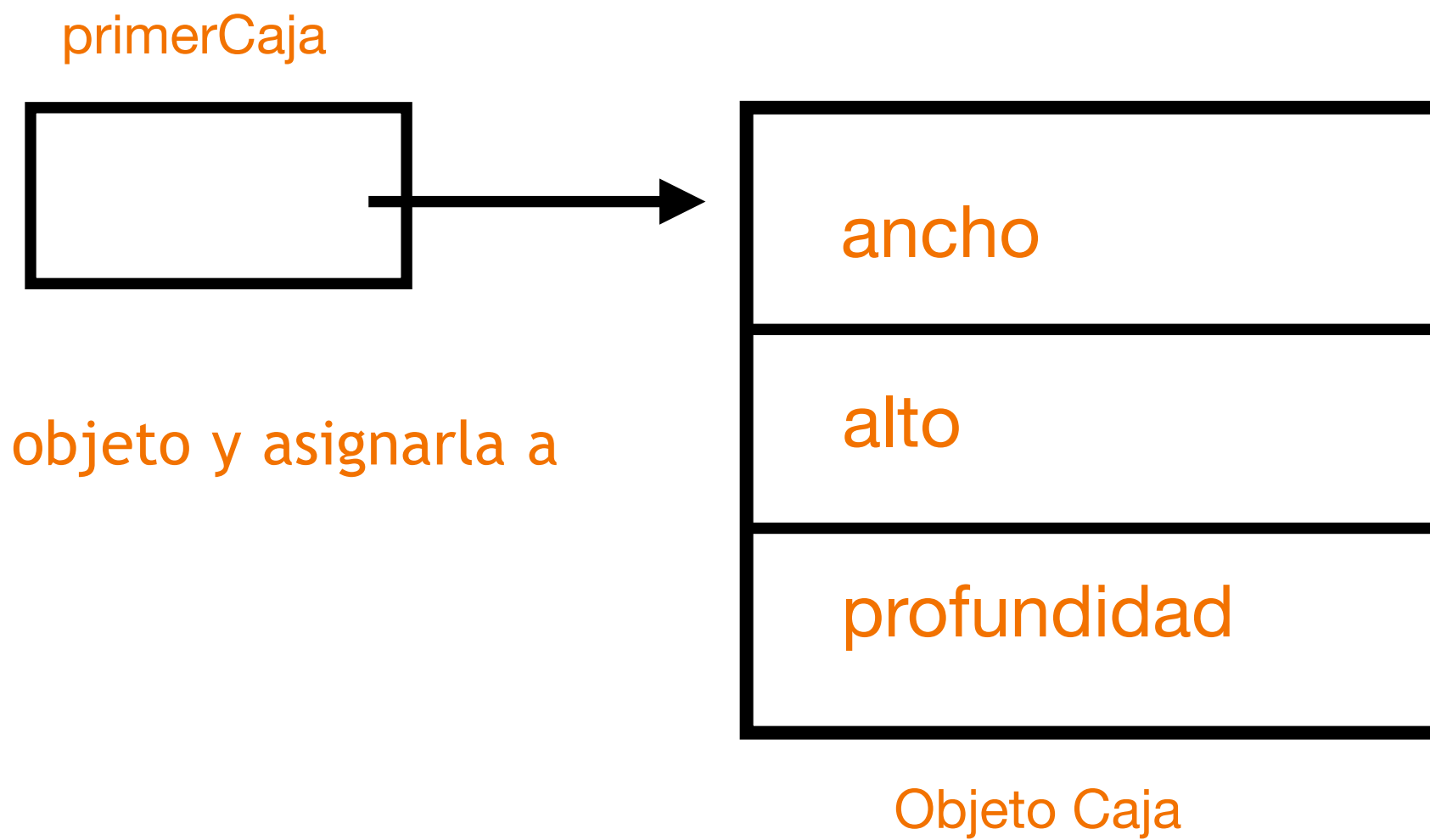
```
public class PruebaCaja {  
    public static void main(String[] args) {  
        Caja miCaja1 = new Caja();  
        Caja miCaja2 = new Caja();  
    }  
}
```

- Para crear objetos de la clase, es necesario crear una **instancia de la Clase**

Declarar objetos

- ▶ **Sintaxis:**
 - ▶ NombreClase variableClase;
- ▶ Primero debe **declarar una variable** del tipo de **clase**. Donde variableClase es una variable del tipo de clase, es decir, se esta creando un nuevo **tipo de datos**. Esta variable no define un objeto.
 - ▶ Ejemplo:
`Caja primerCaja; // declara una referencia al objeto`
- ▶ Puede utilizar este tipo para **declarar objetos** de este tipo.

Crear objetos



- ▶ En segundo lugar, debe adquirir **una copia física real del objeto y asignarla a esa variable**. Utilizando el operador new.
- ▶ Sintaxis:
 - ▶ `variableClase = new Clase();`
- ▶ El **operador new** asigna memoria dinámicamente a un objeto (en tiempo de ejecución) y devuelve la referencia a él.
 - ▶ Esta referencia es, **la dirección en memoria del objeto asignado**. Esta referencia se almacena en la variable.
- ▶ Al **crear una instancia de una clase**, se está llamando a un **método especial de la clase llamado constructor**. Un constructor define qué ocurre cuando se crea un objeto de una clase.
- ▶ Ejemplo:
`primerCaja = new Caja(); //aloca un objeto Caja`
- ▶ Sin embargo, si no se especifica un constructor explícito, Java proporcionará automáticamente uno predeterminado.

Crear objetos de la clase caja

- Acceder a las variables de instancia

```
public class PruebaCaja {  
    public static void main(String[] args) {  
        Caja miCaja1 = new Caja();  
  
        // asignar valores a las variables de instancia de miCaja1  
        miCaja1.ancho = 10;  
        miCaja1.alto = 20;  
        miCaja1.profundo = 15;  
  
        Caja miCaja2 = new Caja();  
        /* asignar valores a las variables de instancia de miCaja2 */  
        miCaja2.ancho = 3;  
        miCaja2.alto = 6;  
        miCaja2.profundo = 9;  
    }  
}
```

- Cuando se **accede a una variable de instancia** por código que no es parte de la clase en la que se define esa variable de instancia, debe hacerse a través de un objeto, mediante el uso **operador punto (DOT)**.

Caja
+ancho: double +altura: double +profundidad: double
+volumen(): void

Objeto miCaja1

miCaja1
ancho: 10 altura: 20 profundidad: 15
volumen():

Objeto miCaja2

miCaja2
ancho: 3 altura: 6 profundidad: 9
volumen():

Crear objetos de la clase caja

► La Ejecución de la clase DemoCaja

```
public class DemoCaja {  
    public static void main(String[] args) {  
        Caja miCaja1 = new Caja();  
  
        // asignar valores a las variables de instancia de miCaja1  
        miCaja1.ancho = 10;  
        miCaja1.alto = 20;  
        miCaja1.profundo = 15;  
  
        // desplegar el volumen de la primer caja llamando el método  
        miCaja1.volumen();  
  
        Caja miCaja2 = new Caja();  
        /* asignar valores a las variables de instancia de miCaja2*/  
        miCaja2.ancho = 3;  
        miCaja2.alto = 6;  
        miCaja2.profundo = 9;  
  
        // desplegar el volumen de la segunda caja llamando el método  
        miCaja2.volumen();  
    }  
}
```

Salida del código

```
Volumen de la caja es 3000.0  
Volumen de la caja es 162.0
```

Operador .

- Una vez que creamos el objeto, para poder **acceder a los atributos y/o métodos** de la clase se utiliza el operador punto.

Método

► Forma general de un método:

Tipo de función

Parametros o argumentos

```
tipoDato nombreMetodo(tipoDato parametro1, tipoDato2 parametro2, ...,)  
{  
    return valorRetorno;  
}
```

Valor de retorno

Cuerpo del método

- `tipoDato` especifica el tipo de datos que devuelve el método. Puede ser cualquier tipo válido incluido los tipos de clase que crees. Si el método no devuelve un valor, su tipo debe ser `void`

Método

► Forma general de un método:

```
Tipo de función  
tipoDato nombreMetodo(Parametros o argumentos  
{ tipoDato parametro1, tipoDato2 parametro2, ..., )  
  
[ return valorRetorno;  
] Valor de retorno  
  
Cuerpo del método
```

- El nombre del método se especifica, puede ser cualquier identificador valido.
- La lista de parámetros es una secuencia de pares de tipo e identificador separados por comas. Los parámetros son, variables que reciben el valor de los argumentos pasados al método cuando se llama.
- Si el método no tiene parámetros, la lista de parámetros estará vacía.

Método

- Es una colección de instrucciones (statements) que realizan una tarea específica.
- El enfoque de dividir un programa en partes se denomina a veces “divide y vencerás”.

```
public class BigProblem
{
    public static void main(String[] args)
    {
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
        statement;
    }
}
```

```
public class DividedProblem
{
    public static void main(String[] args)
    {
        statement;
        statement;
        statement;
    }

    public static void method2()
    {
        statement;
        statement;
        statement;
    }

    public static void method3()
    {
        statement;
        statement;
        statement;
    }

    public static void method4()
    {
        statement;
        statement;
        statement;
    }
}
```

Parámetros de los métodos

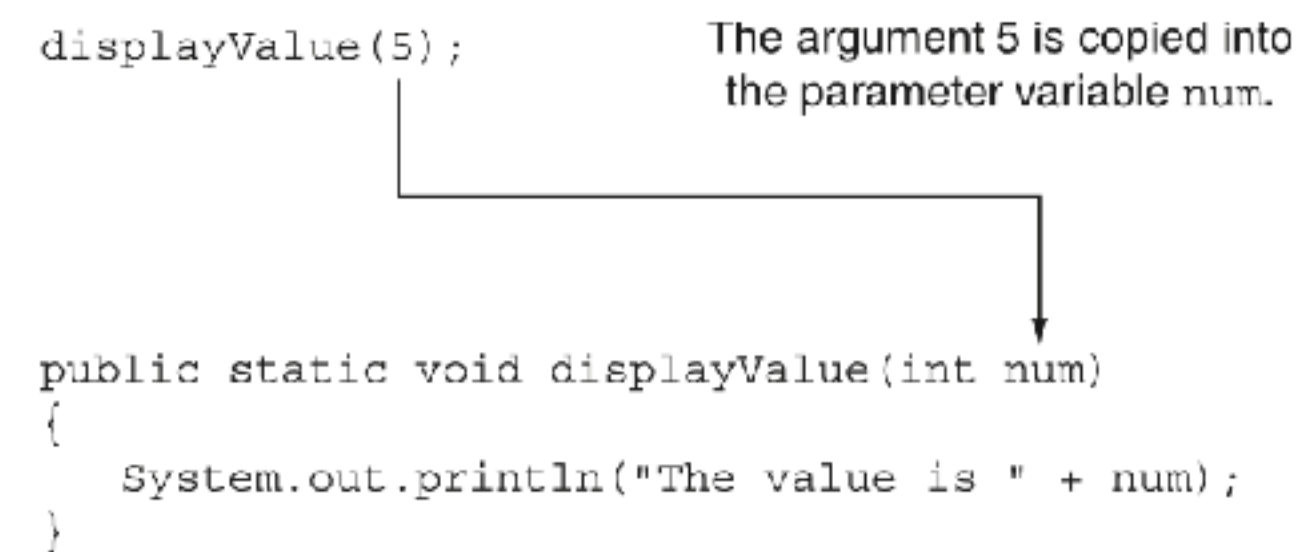
- ▶ Existen dos formas de pasar valores a un método
 - ▶ La primera forma es cuando los valores se copian, se le llama **pase por valor**, los cambios solo existen en el bloque de código del método.
 - ▶ La segunda es cuando la pasamos **por referencia (dirección de memoria)** por lo que los valores no se copian y **accedes directo a ellos**, por lo que los cambios persisten fuera del bloque de código del método.

Parámetros de los métodos

- ▶ En Java siempre es pase por valor (copia) sin embargo eso va a depender si el parámetro es una variable tipo primitivo (int, float, etc) o un objeto.
- ▶ Si es un objeto entonces es pace por referencia ya que el objeto es una referencia a memoria de la clase

Pase de parametros

- ▶ Un parámetro, es una **variable definida por un método que recibe un valor cuando se invoca dicho método..**
- ▶ Un argumento es un valor que se pasa a un método cuando se invoca.



The diagram illustrates the process of passing an argument to a method. It shows a method call `displayValue(5);` at the top. A line connects the number `5` to the parameter `num` in the method signature `public static void displayValue(int num)` below. An arrow points from the `5` to the `num` parameter. A text box above the arrow states: "The argument 5 is copied into the parameter variable num."

```
displayValue(5);
```

The argument 5 is copied into the parameter variable num.

```
public static void displayValue(int num)
{
    System.out.println("The value is " + num);
}
```

- ▶ El argumento que se encuentra dentro del paréntesis se copia en la variable de parámetro del método, num.

Pase múltiple de parametros

- Pasar más de un argumento a un método, esto suele denominarse **lista de parámetros**, los cuales deben separarse con una coma.

The argument 5 is copied into the `num1` parameter.
The argument 10 is copied into the `num2` parameter.

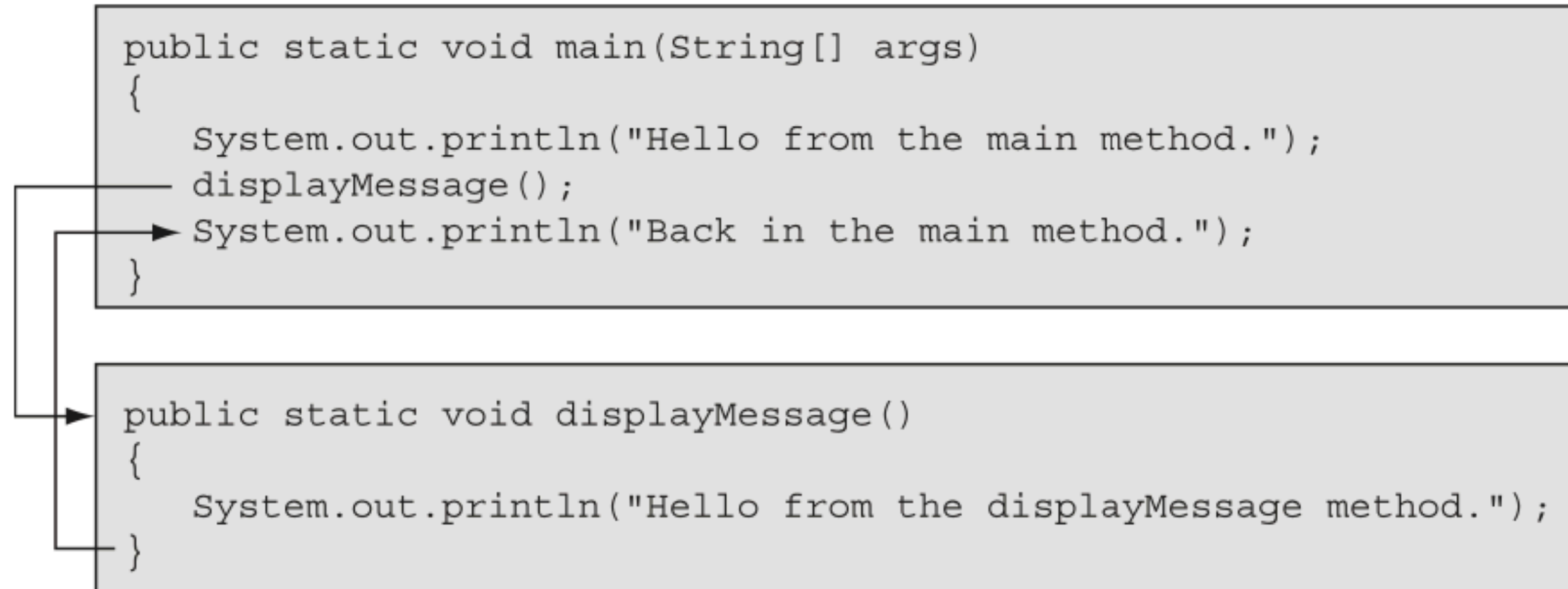
```
showSum(5, 10);
```

```
public static void showSum(double num1, double num2)
{
    double sum;    // To hold the sum

    sum = num1 + num2;
    System.out.println("The sum is " + sum);
}
```

Llamada a un método

- Un método se ejecuta cuando **se le llama o invoca**. El método main se llama automáticamente cuando se inicia un programa, pero los demás métodos se ejecutan mediante instrucciones de llamada de métodos.



Método con valor de retorno

- ▶ Un **método que devuelve un valor** no solo realiza una tarea, sino que también envía un valor al código que lo llamó. El método se ejecuta y luego devuelve el valor.
- ▶ Cuando utilizas un método que devuelven un valor, debes **decidir qué tipo de valor devolverá**. Esto se debe especificar en el encabezado del método.

Return Type
↓
`public static int sum(int num1, int num2)`

- ▶ Este código define un método llamado suma que acepta dos parámetros int. Los argumentos se pasan a las variables de parámetro num1, num2.

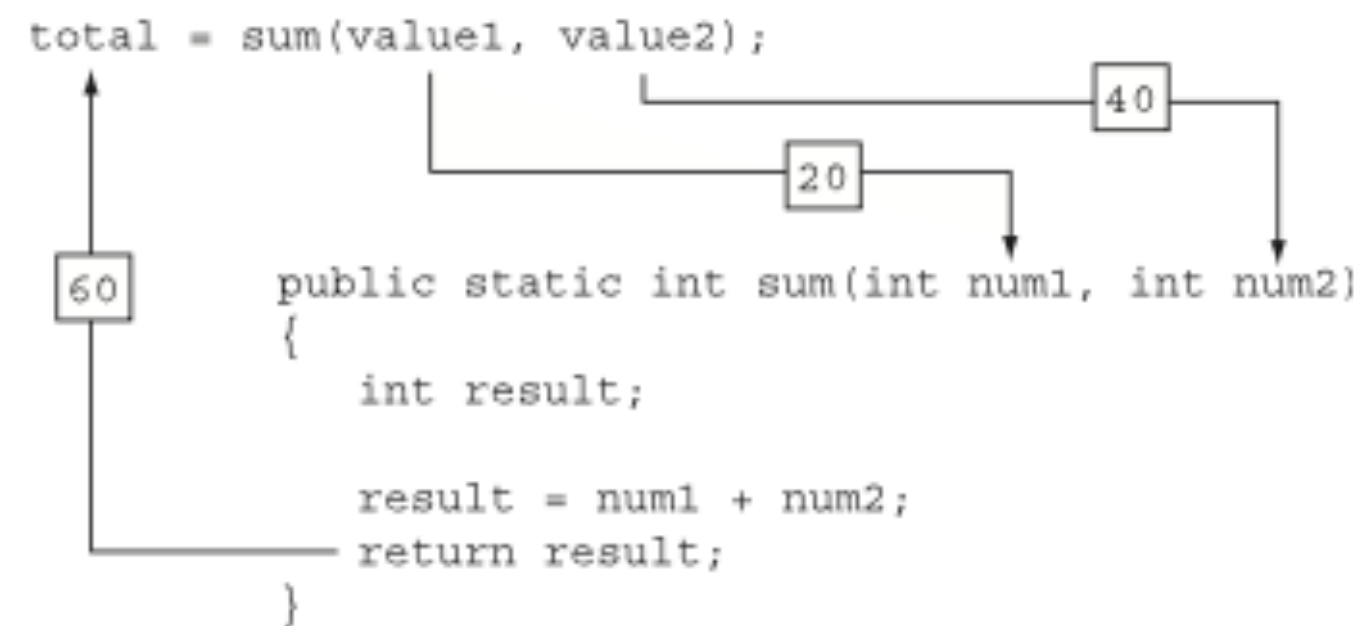
Retorno de un valor desde un método

- Dentro del método o cuerpo del método se suman y se asigna el resultado a la variable suma, la ultima instrucción es la declaración de retorno. Todo método que regresa valor debe tener una declaración de retorno

```
public static int sum(int num1, int num2)
{
    return num1 + num2;
}
```

Llamada a un método que devuelve un valor

- Cuando se llama a un método que devuelve un valor, normalmente se desea hacer algo significativo con el valor que devuelve.



- Así es como se utilizan habitualmente los valores de retorno, pero se pueden hacer muchas otras cosas con ellos.

```
int x = 10, y = 15;
double average;
average = sum(x, y) / 2.0;
```

```
int x = 10, y = 15;
System.out.println("The sum is " + sum(x, y));
```


Ejemplo de clase: Mejora 2

// Este programa usa un método para asignar valores a las variables de instancia

```
class Caja3 {  
    double ancho;  
    double altura;  
    double profundidad;  
  
    double volumen() {  
        return ancho * altura * profundidad;  
    }  
    // Asigna las dimensiones de la caja  
    void setDim(double w, double h, double d) {  
        ancho = w;  
        altura = h;  
        Profundidad = d; }  
}
```

- Por qué el volumen no lo guardamos en un atributo?

Caja3
+ancho: double +altura: double +profundidad: double +vol: double
+volumen(): double +setDim(double, double, double):void



Clase DemoCaja: Mejora 2

```
public class DemoCaja3 {
    public static void main(String[] args) {
        Caja3 miCaja1 = new Caja3();
        Caja3 miCaja2 = new Caja3();
        double vol;

        miCaja1.setDim(10,20,15);
        miCaja2.setDim(3,6,9);

        // obtiene el volumen de la primera caja
        vol = miCaja1.volumen();
        System.out.println("Volumen de caja 1 es "+ vol);
        // obtiene el volumen de la segunda caja
        vol = miCaja2.volumen();
        System.out.print("Volumen de caja 2 es "+ vol);

    }
}
```

Salida del código

```
El volumen de caja 1 es: 3000.0
El volumen de caja 2 es: 162.0
```

Objeto miCaja1

miCaja1
Ancho: 10 Altura: 20 Profundidad: 15 Vol: 3000
volume(): setDim(Ancho,Altura,Profundidad)

Objeto miCaja2

miCaja2
Ancho: 3 Altura: 6 Profundidad: 9 Vol: 162
volume(): setDim(Ancho,Altura,Profundidad)

Definición de la Clase Alumno

// Definición de la clase Alumno

```
public class Alumno {  
  
    double matricula;  
    String nombreAlumno;  
    String carrera;  
    int semestre, diaNac, mesNac, anioNac;  
  
    int calcularEdad(int anioNac) {  
        // regresa la edad del alumno  
        return 2024-anioNac;  
    }  
}
```

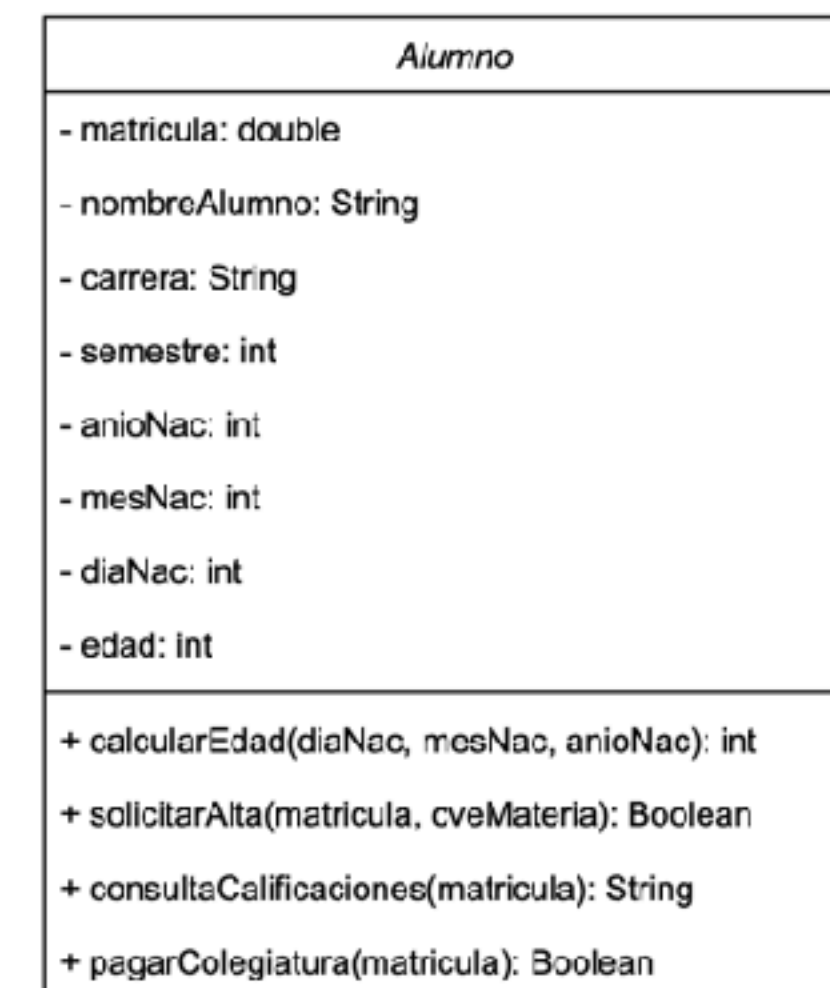
Alumno
- matricula: double - nombreAlumno: String - carrera: String - semestre: int - anioNac: int - mesNac: int
+ calcularEdad(diaNac, mesNac, anioNac): int

Crear objetos

- Representación de la creación de objetos a partir de clases de forma esquemática

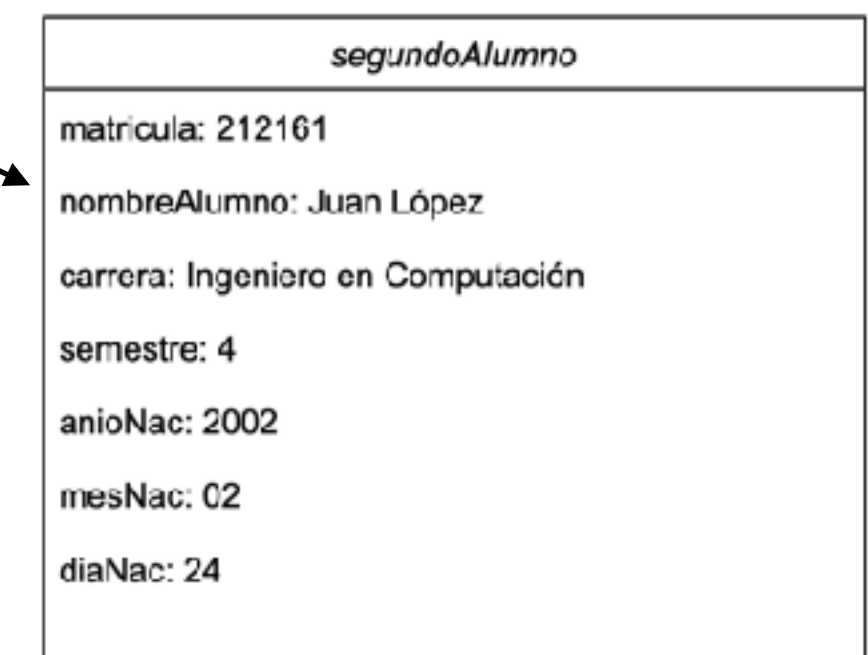
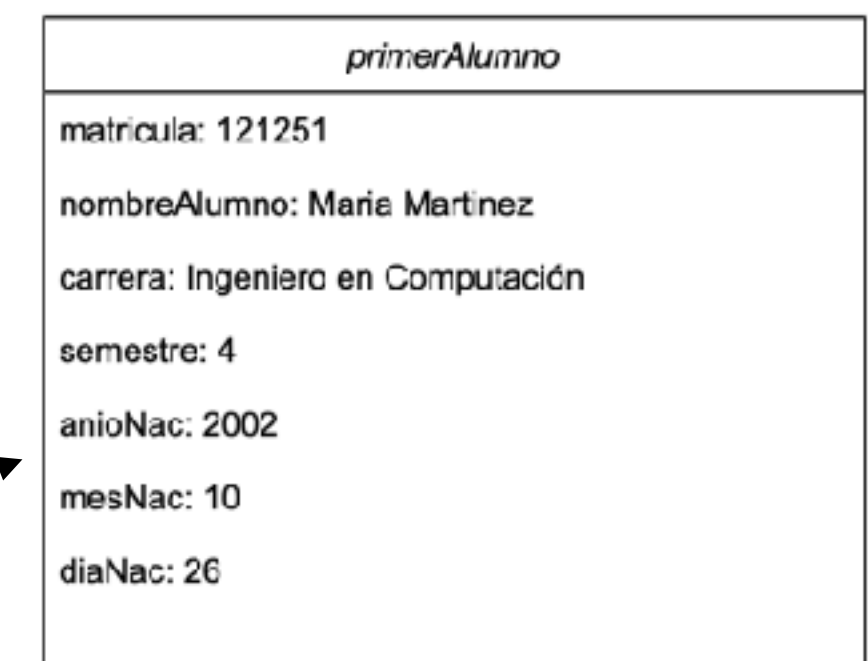
// Definición de la clase PruebaEstudiante

```
public class PruebaAlumno {  
    public static void main(String[] args) {  
        Alumno primerAlumno = new Alumno();  
        Alumno segundoAlumno = new Alumno();  
        primerAlumno.matricula = 121251;  
        primerAlumno.nombreAlumno = "Maria Martinez";  
        primerAlumno.carrera = "Ingeniero en Computacion";  
        primerAlumno.semestre = 4;  
        primerAlumno.anioNac = 2002;  
        primerAlumno.mesNac = 10;  
        segundoAlumno.diaNac = 15;  
  
        segundoAlumno.matricula = 212161;  
        segundoAlumno.nombreAlumno = "Juan Lopez";  
        segundoAlumno.carrera = "Ingeniero en Computacion";  
        segundoAlumno.semestre = 4;  
        segundoAlumno.anioNac = 2002;  
        segundoAlumno.mesNac = 02;  
        segundoAlumno.diaNac = 16;  
    }  
}
```



new

new



Estado del objeto

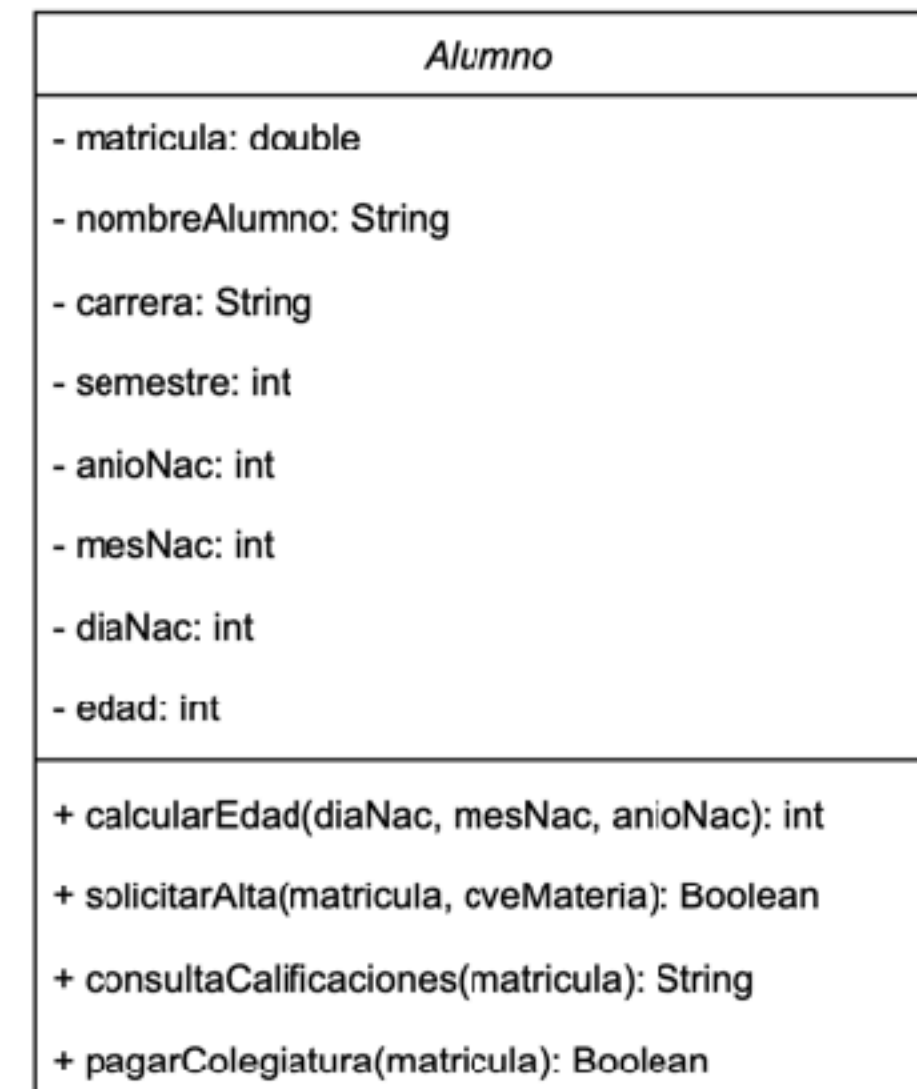
```
// Definición de la clase PruebaEstudiante
```

```
public class PruebaAlumno {  
    public static void main(String[] args) {  
        Alumno primerAlumno = new Alumno();  
        Alumno segundoAlumno = new Alumno();  
        primerAlumno.matricula = 121251;  
        primerAlumno.nombreAlumno = "Maria Martinez";  
        primerAlumno.carrera = "Ingeniero en Computacion";  
        primerAlumno.semestre = 4;  
        primerAlumno.anioNac = 2002;  
        primerAlumno.mesNac = 10;  
        primerAlumno.diaNac = 26;
```

```
        segundoAlumno.matricula = 212161;  
        segundoAlumno.nombreAlumno = "Juan Lopez";  
        segundoAlumno.carrera = "Ingeniero en Computacion";  
        segundoAlumno.semestre = 4;  
        segundoAlumno.anioNac = 2002;  
        segundoAlumno.mesNac = 02;  
        segundoAlumno.diaNac = 24;
```

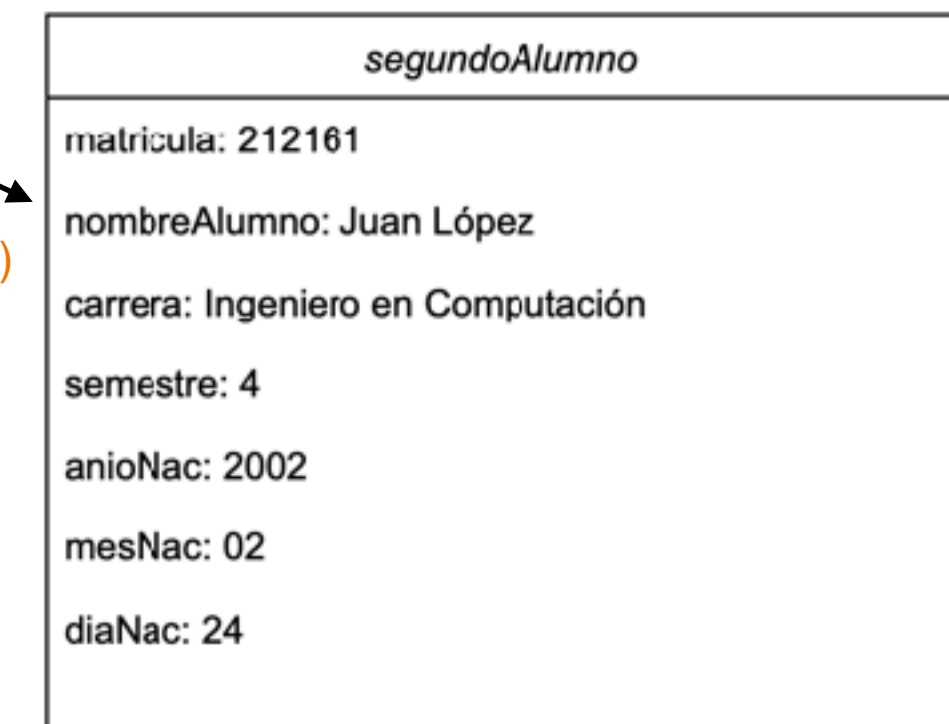
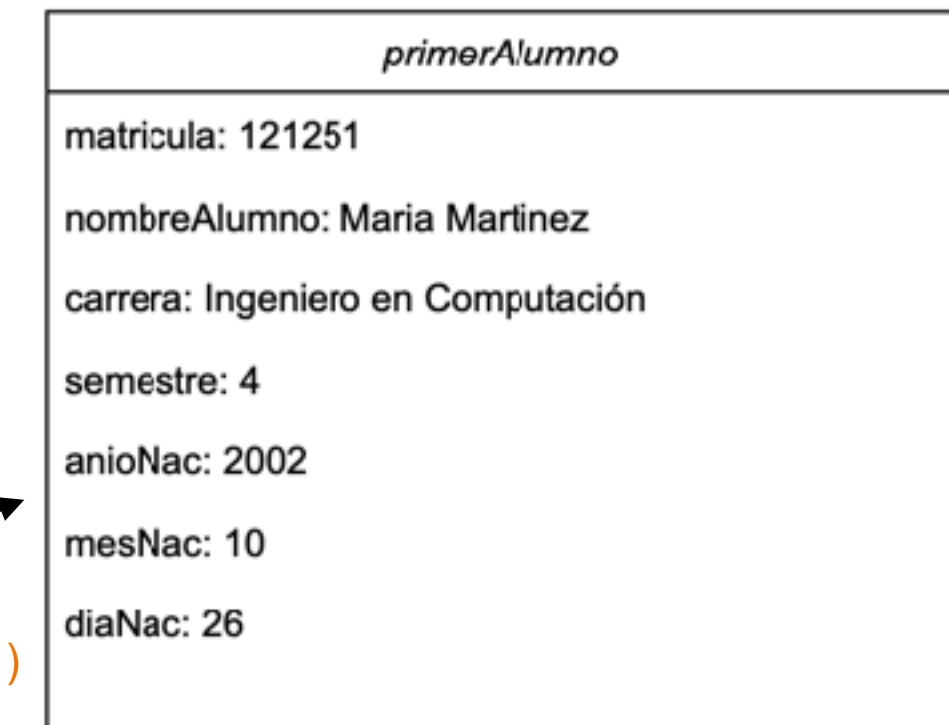
```
        System.out.println("La matricula del alumno 1 es:" + primerAlumno.matricula);  
        System.out.println("El nombre del estudiante es " + primerAlumno.nombreAlumno);  
        System.out.println("La carrera que cursa es " + primerAlumno.carrera);  
        System.out.println("El semestre que estas cursando es " + primerAlumno.semestre);
```

```
    }  
}
```



new Alumno()

new Alumno()



Invocar métodos

// Definición de la clase PruebaEstudiante

```
public class PruebaAlumno {
    public static void main(String[] args) {
        Alumno primerAlumno = new Alumno();
        Alumno segundoAlumno = new Alumno();
        primerAlumno.matricula = 121251;
        primerAlumno.nombreAlumno = "Maria Martinez";
        primerAlumno.carrera = "Ingeniero en Computacion";
        primerAlumno.semestre = 4;
        primerAlumno.anioNac = 2002;
        primerAlumno.mesNac = 10;
        primerAlumno.anioNac = 2004;

        segundoAlumno.matricula = 212161;
        segundoAlumno.nombreAlumno = "Juan Lopez";
        segundoAlumno.carrera = "Ingeniero en Computacion";
        segundoAlumno.semestre = 4;
        segundoAlumno.anioNac = 2002;
        segundoAlumno.mesNac = 02;
        segundoAlumno.anioNac = 2003;

        System.out.println("La matricula del alumno 1 es:" + primerAlumno.matricula);
        System.out.println("El nombre del estudiante es " + primerAlumno.nombreAlumno);
        System.out.println("La carrera que cursa es " + primerAlumno.carrera);
        System.out.println("El semestre que estas cursando es " + primerAlumno.semestre);

        // invocar el método calcular edad
        System.out.println("La edad del Alumno es:" + primerAlumno.calcularEdad(primerAlumno.anioNac));
    }
}
```


Atributos desactualizados

- ▶ Se dice que un atributo esta **viejo (stale)** cuando **depende de otros atributos**, como en el caso de volumen, depende de otros atributos.
- ▶ Si uno de los otros atributos cambia, el volumen **no cambiaría y se volvería un valor desactualizado**.